

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
**TRƯỜNG ĐIỆN - ĐIỆN TỬ**



**BÁO CÁO ĐỒ ÁN II**

*Đề tài:* **THIẾT KẾ HỆ THỐNG ĐO NHIỆT ĐỘ ĐỘ ẨM HIỂN  
THỊ TRÊN MÀN HÌNH LCD**

*Giảng viên hướng dẫn:* **Thầy Dương Thanh Phương**

*Sinh viên thực hiện:* **Vũ Ngọc Duẩn \_ 20213832**

*Hà Nội, 2025*



**BẢNG PHÂN CÔNG NHIỆM VỤ**

| Họ và tên    | Nhiệm vụ                                                                                      | Tự đánh giá               | Ghi chú |
|--------------|-----------------------------------------------------------------------------------------------|---------------------------|---------|
| Vũ Ngọc Duẩn | Hàn mạch, mô phỏng proteus, đo kiểm tra hoạt động của mạch, tìm hiểu code, tìm kiếm linh kiện | Hoàn thành phần được giao |         |

# LỜI MỞ ĐẦU

Ngày nay, khi khoa học công nghệ đang phát triển mạnh mẽ, việc giám sát các thông số môi trường như nhiệt độ và độ ẩm ngày càng trở nên quan trọng, đặc biệt trong các lĩnh vực đời sống, sản xuất và nông nghiệp. Trong đó, cảm biến DHT11 là một trong những giải pháp phổ biến giúp đo nhiệt độ và độ ẩm với chi phí thấp, dễ sử dụng và tích hợp cao. Với sự hỗ trợ từ vi điều khiển ATmega16, các dữ liệu đo được có thể xử lý và hiển thị trực tiếp lên màn hình LCD 16x2, tạo nên một hệ thống giám sát môi trường đơn giản nhưng hiệu quả.

Việc hiển thị thông tin từ cảm biến lên LCD giúp người dùng dễ dàng theo dõi điều kiện môi trường theo thời gian thực, từ đó ứng dụng cho các mục đích như điều khiển hệ thống thông gió, giám sát nhiệt độ phòng, nhà kho, nhà kính hoặc các ứng dụng giáo dục và nghiên cứu. Đây cũng chính là ý tưởng để em thực hiện đề tài thiết kế "**Mạch hiển thị nhiệt độ và độ ẩm sử dụng cảm biến DHT11 và LCD 16x2**".

Em xin được gửi lời cảm ơn chân thành đến các Thầy Cô trong Viện Điện tử – Viễn thông đã tận tình giảng dạy, truyền đạt cho em những kiến thức nền tảng và chuyên ngành quý báu trong suốt thời gian học tập vừa qua. Đặc biệt, em xin gửi lời cảm ơn sâu sắc tới thầy **Dương Thanh Phương** vì sự hướng dẫn tận tình và luôn tạo điều kiện thuận lợi để em có thể thực hiện và hoàn thành tốt đề tài này.

Mặc dù nhóm đã rất cố gắng và nỗ lực trong quá trình thực hiện, nhưng do kiến thức và kinh nghiệm còn hạn chế, không thể tránh khỏi những thiếu sót trong quá trình làm đề tài. Em rất mong nhận được sự góp ý, nhận xét từ Thầy để em có thể rút kinh nghiệm và hoàn thiện tốt hơn trong các đề tài tiếp theo.

**Em xin chân thành cảm ơn!**

Hà Nội, năm 2025

*Sinh viên thực hiện*

Duẩn

Vũ Ngọc Duẩn

# MỤC LỤC

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| DANH MỤC HÌNH VẼ .....                                                              | i  |
| PHẦN MỞ ĐẦU .....                                                                   | ii |
| CHƯƠNG 1: CƠ SỞ LÝ THUYẾT TỔNG QUAN .....                                           | 1  |
| 1.1.    Tìm hiểu vi điều khiển Atmega16 .....                                       | 1  |
| 1.1.1.    Tổng quan về Atmega16 .....                                               | 1  |
| 1.1.2.    Cấu trúc Atmega16.....                                                    | 3  |
| 1.1.3.    ARV CPU Core .....                                                        | 4  |
| 1.1.4.    Các cổng I/O (xuất nhập) .....                                            | 7  |
| 1.1.5.    Bộ định thời, bộ đếm Timer/Counter .....                                  | 9  |
| 1.2.    PWM(Pulse Width Modulation)- điều chế độ rộng xung.....                     | 11 |
| 1.2.1.    Tổng quan về PWM .....                                                    | 11 |
| 1.2.2.    Điều chế xung PWM bằng chế độ Fast PWM của vi điều khiển<br>Atmega16..... | 12 |
| 1.3.    Các phần mềm sử dụng.....                                                   | 14 |
| 1.3.1.    Trình biên dịch CodevisionAVR.....                                        | 14 |
| 1.3.2.    Phần mềm mô phỏng Proteus.....                                            | 15 |
| 1.3.3.    Phần mềm nạp code ProgISP .....                                           | 15 |
| 1.4.    Các linh kiện được sử dụng .....                                            | 16 |
| 1.4.1.    Cảm biến DHT11 .....                                                      | 16 |
| 1.4.2.    Màn hình hiển thị LCD .....                                               | 17 |
| CHƯƠNG 2: THIẾT KẾ MẠCH.....                                                        | 18 |
| 2.1. Sơ đồ khối .....                                                               | 18 |
| 2.2. Mạch in.....                                                                   | 18 |
| CHƯƠNG 3: YÊU CẦU MẠCH .....                                                        | 19 |
| 3.1. Yêu cầu chức năng .....                                                        | 19 |
| 3.2. Yêu cầu phi chức năng .....                                                    | 19 |
| 3.3. Ứng dụng.....                                                                  | 19 |
| CHƯƠNG 4: KẾT QUẢ THỰC HIỆN.....                                                    | 20 |

|                                      |    |
|--------------------------------------|----|
| 4.1. Kết quả mô phỏng .....          | 20 |
| 4.2. Kết quả thực tế trên mạch ..... | 20 |
| KẾT LUẬN .....                       | 22 |
| PHỤ LỤC .....                        | 23 |
| Phụ lục 1: Source code .....         | 23 |
| File <main.c> .....                  | 23 |
| File <LCD.h> .....                   | 24 |
| File <DHT11.c> .....                 | 25 |
| File <LCD.c > .....                  | 26 |
| File <DHT11.h > .....                | 27 |

# DANH MỤC HÌNH VẼ

|                                                         |    |
|---------------------------------------------------------|----|
| Hình 1. 1 Cấu trúc các khối Atmega16 .....              | 3  |
| Hình 1. 2 Sơ đồ khối AVR MCU .....                      | 4  |
| Hình 1. 3 Các thanh ghi chức năng chung.....            | 5  |
| Hình 1. 4 Cấu hình chân Atmega16 .....                  | 7  |
| Hình 1. 5 Cấu trúc khối timer/counter1.....             | 9  |
| Hình 1. 6 Khối bộ đếm .....                             | 10 |
| Hình 1. 7 Đồ thị dạng xung điều chế PWM .....           | 11 |
| Hình 1. 8 Các chế độ hoạt động của timer/counter1 ..... | 12 |
| Hình 1. 9 Mô tả hoạt động Fast PWM.....                 | 12 |
| Hình 1. 10 Giao diện của CodevisionAVR .....            | 14 |
| Hình 1. 11 Giao diện Proteus .....                      | 15 |
| Hình 1. 12 Giao diện ProgISP 1.72 .....                 | 16 |
| Hình 1. 13 Cảm biến DHT11 .....                         | 17 |
| Hình 1. 14 LCD 16x2 .....                               | 17 |
| <br>                                                    |    |
| Hình 2. 1 Sơ đồ khối.....                               | 18 |
| Hình 2. 2 Mạch in PCB .....                             | 18 |
| <br>                                                    |    |
| Hình 4. 1 Kết quả mô phỏng bằng phần mềm Proteus.....   | 20 |
| Hình 4. 2 Kết quả thực tế.....                          | 21 |

## PHẦN MỞ ĐẦU

Đối với sinh viên học ngành điện tử viễn thông, việc lập trình, thiết kế và làm mạch là một kỹ năng cơ bản không thể thiếu trong ngành nghề của mình. Học phần đồ án thiết kế II là cơ hội để mỗi sinh viên có thể luyện tập, mài dũa kỹ năng đó.

Sau thời gian học tập 3 năm tại trường, được sự chỉ bảo hướng dẫn của các thầy cô trong viên Điện tử Viễn thông, chúng em đã tích lũy được vốn kiến thức nhất định. Được sự đồng ý của thầy Dương Thanh Phương, chúng em được giao đề tài nghiên cứu đồ án thiết kế II: “*Mạch đo nhiệt độ độ ẩm sử dụng DHT11 hiển thị lên LCD 16x2*”.

Đồ án thiết kế II của em gồm 4 chương:

Chương 1: Trình bày cơ sở lý thuyết Atmega16, các linh kiện sử dụng, các phần mềm sử dụng

Chương 2: Thiết kế mạch

Chương 3: Yêu cầu mạch

Chương 4: Kết quả thực hiện

Trong quá trình thực hiện đề tài, do còn nhiều hạn chế về mặt thời gian, kiến thức cũng như công cụ và kinh nghiệm nên đề tài vẫn còn những thiếu sót nhất định, mong được thầy cô và các bạn đánh giá, góp ý một cách khách quan để chúng em có thể rút ra kinh nghiệm và hoàn thiện đề tài hơn nữa. Để hoàn thành bài báo cáo này, chúng em xin chân thành gửi lời cảm ơn tới sự quan tâm và hướng dẫn tận tình của Thầy.

# CHƯƠNG 1: CƠ SỞ LÝ THUYẾT TỔNG QUAN

Ở chương này chúng em chủ yếu nói về cấu trúc, một số chức năng chính của vi điều khiển ATMEGA16 (họ vi điều khiển AVR) sử dụng trong đề tài và các phần mềm, linh kiện được sử dụng

## 1.1. Tìm hiểu vi điều khiển Atmega16

### 1.1.1. Tổng quan về Atmega16

Atmega16 là họ IC vi điều khiển do hãng Atmel sản xuất. Các sản phẩm Atmega thích hợp cho những ứng dụng điều khiển. Việc xử lý trên byte và các phép toán số học ở cấu trúc dữ liệu nhỏ được thực hiện bằng nhiều chế độ truy xuất dữ liệu nhanh trên RAM nội. Tập lệnh cung cấp một bảng tiện dụng của những lệnh số học 8 bit gồm cả lệnh nhân và lệnh chia. Nó cung cấp những hỗ trợ mở rộng trên chip dùng cho những biến một bit như là kiểu dữ liệu riêng biệt cho phép quản lý và kiểm tra bit trực tiếp trong hệ thống điều khiển.

Các đặc điểm của chip Atmega được tóm tắt như sau:

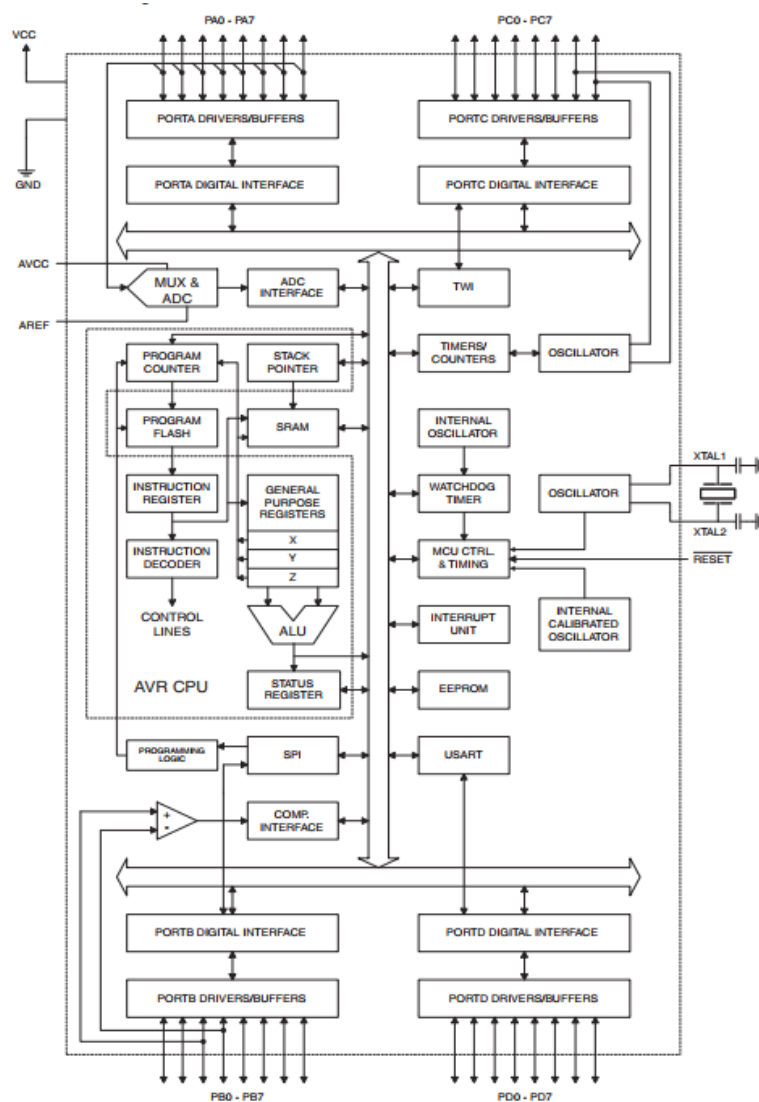
- Hiệu suất cao ( high performance ), là loại vi điều khiển AVR 8 bit công suất thấp
- Cấu trúc lệnh đơn giản, thời gian thực thi lệnh như nhau ( thật ra là Advanced RISC Architecture )
  - 130 lệnh thực thi trong vòng 1 chu kì chip
  - 32 x 8 thanh ghi công dụng chung ( chắc là 32 thanh ghi công dụng chung 8 bit )
  - Đầy đủ các xử lý tĩnh
  - Hỗ trợ 16 MIPS khi hoạt động ở tần số 16 MHz
  - Tích hợp bộ nhân 2 thực hiện trong 2 chu kì chip
- Bộ nhớ chương trình và dữ liệu không bay hơi ( nonvolatile )
  - 16k byte trong hệ thống flash khả trình có thể nạp và xóa 1,000 lần
  - Tùy chọn khởi động phần mã với các bit nhìn độc lập trong hệ thống bằng cách vào chương trình khởi động chip
  - 512 byte EEPROM có thể ghi và xóa 100,000 lần
  - 1k byte ram nhớ tĩnh trong ( internal SRAM )
  - Lập trình khóa cho phần mềm bảo mật

- Tính năng ngoại vi
  - 2 bộ định thời/bộ đếm ( timers/counters ) 8 bit với các chế độ đếm riêng rẽ và kiểu so sánh
  - 1 bộ định thời/bộ đếm ( timer/counter ) 16 bit với các chế độ đếm riêng rẽ, kiểu so sánh và kiểu bắt sự kiện
  - Bộ đếm thời gian thực với máy giao động riêng rẽ
  - 4 kênh băm xung PWM
  - 8 kênh ADC 10 bit
  - Byte định hướng 2 đường giao tiếp nối tiếp
  - Giao tiếp USART nối tiếp khả trình
  - Giao tiếp SPI nối tiếp chủ/tớ ( master/slave )
  - Bộ định thời khả trình giám sát xung nhịp của chip 1 cách riêng rẽ
  - Tích hợp bộ so sánh tín hiệu tương tự
- Giao tiếp JTAG
- Các tính năng đặc biệt của vi điều khiển
  - Chế độ bật nguồn reset và phát hiện Brown-out khả trình
  - Tích hợp mạch dao động RC bên trong
  - Các ngắt trong và ngoài
  - 6 chế độ nghỉ : rảnh rỗi, giảm nhiễu ADC, Tiết kiệm năng lượng, nguồn thấp, Standby và Extended Standby
- Vào/ra và các gói dữ liệu
  - 32 chân vào ra khả trình
  - 40-pin PDIP and 44-lead TQFP
- Điện áp sử dụng
  - 2.7 – 5.5V dùng với atmega16L
  - 4.5 – 5.5V dùng với atmega16
- Tốc độ xung nhịp dùng cho chip
  - 0 – 8 MHz cho atmega16L
  - 0 – 16 MHz cho atmega16

### 1.1.2. Cấu trúc Atmega16

ATmega16 là vi điều khiển CMOS 8-bit công suất thấp dựa trên lõi AVR nâng cao.

Kiến trúc RISC. Bằng cách thực hiện các hướng dẫn mạnh mẽ trong một chu kỳ đồng hồ, ATmega16 đạt được thông lượng lên tới gần 1 MIPS trên mỗi MHz cho phép hệ thống thiết kế để tối ưu hóa mức tiêu thụ điện năng so với tốc độ xử lý.

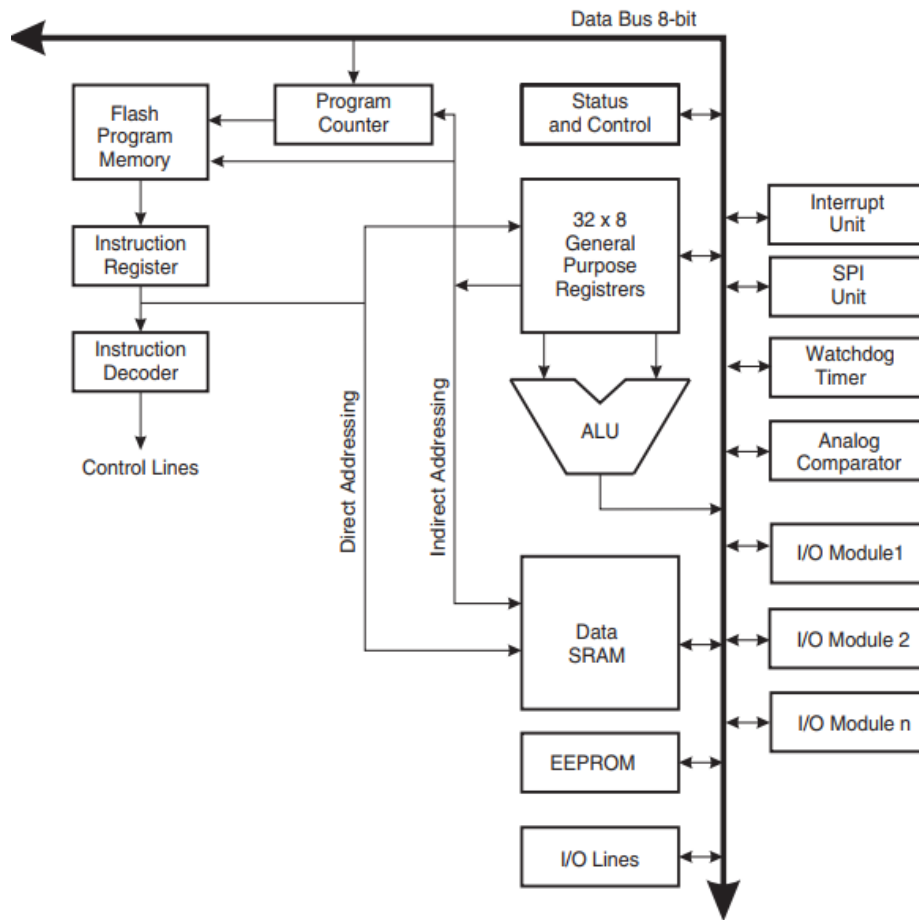


**Hình 1. 1 Cấu trúc các khối Atmega16**

Lõi AVR kết hợp một tập lệnh phong phú với 32 thanh ghi làm việc cho mục đích chung. Tất cả 32 thanh ghi được kết nối trực tiếp với đơn vị Logic số học (ALU), cho phép hai thanh ghi độc lập được truy cập trong một lệnh duy nhất được thực thi trong một đồng hồ đi xe đạp. Cấu trúc này nhanh hơn mười ngàn lần CISC

### 1.1.3. ARV CPU Core

Lỗi CPU đảm bảo thực hiện đúng chương trình. Do đó, CPU phải có khả năng truy cập bộ nhớ, thực hiện các phép tính, điều khiển thiết bị ngoại vi và xử lý ngắt.



Hình 1. 2 Sơ đồ khối AVR MCU

### ALU

ALU làm việc trực tiếp với các thanh ghi chức năng chung. Các phép toán được thực hiện trong một chu kỳ xung clock. Hoạt động của ALU được chia thành 3 loại: đại số, logic, theo bit.

## Thanh ghi trạng thái

Thanh ghi trạng thái AVR-SREG được định nghĩa:

| Bit           | 7   | 6   | 5   | 4   | 3   | 2   | 1   | 0   |      |
|---------------|-----|-----|-----|-----|-----|-----|-----|-----|------|
|               | I   | T   | H   | S   | V   | N   | Z   | C   | SREG |
| Read/Write    | R/W | R/W | R/W | R/W | R/W | R/W | R/W | R/W |      |
| Initial Value | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   |      |

- **Bit 7-I:** cho phép ngắt toàn cục
- **Bit 6-T:** bộ nhớ sao chép bit
- **Bit 5-H:** cờ half carry
- **Bit 4-S:** bit dấu,  $S=N \text{ XOR } V$
- **Bit 3-V:** bổ sung 2 cờ tràn
- **Bit 2-N:** cờ Negative
- **Bit 1-Z:** cờ zero
- **Bit 0-C:** cờ carry

## Các thanh ghi chức năng chung

|                                            | 7   | 0 | Addr. |                      |
|--------------------------------------------|-----|---|-------|----------------------|
| General<br>Purpose<br>Working<br>Registers | R0  |   | \$00  |                      |
|                                            | R1  |   | \$01  |                      |
|                                            | R2  |   | \$02  |                      |
|                                            | ... |   |       |                      |
|                                            | R13 |   | \$0D  |                      |
|                                            | R14 |   | \$0E  |                      |
|                                            | R15 |   | \$0F  |                      |
|                                            | R16 |   | \$10  |                      |
|                                            | R17 |   | \$11  |                      |
|                                            | ... |   |       |                      |
|                                            | R26 |   | \$1A  | X-register Low Byte  |
|                                            | R27 |   | \$1B  | X-register High Byte |
|                                            | R28 |   | \$1C  | Y-register Low Byte  |
|                                            | R29 |   | \$1D  | Y-register High Byte |
|                                            | R30 |   | \$1E  | Z-register Low Byte  |
|                                            | R31 |   | \$1F  | Z-register High Byte |

Hình 1. 3 Các thanh ghi chức năng chung

## Con trỏ ngăn xếp

Là thanh ghi 16-bit nhưng cũng có thể được coi như là hai thanh ghi 8-bit. Có địa chỉ trong các thanh ghi chức năng đặc biệt như là \$3E ( trong bộ nhớ ram là \$5E) và có nhiệm vụ trỏ tới bộ nhớ RAM chứa ngăn xếp

| Bit           | 15   | 14   | 13   | 12   | 11   | 10   | 9   | 8   |     |
|---------------|------|------|------|------|------|------|-----|-----|-----|
|               | SP15 | SP14 | SP13 | SP12 | SP11 | SP10 | SP9 | SP8 | SPH |
|               | SP7  | SP6  | SP5  | SP4  | SP3  | SP2  | SP1 | SP0 | SPL |
|               | 7    | 6    | 5    | 4    | 3    | 2    | 1   | 0   |     |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W | R/W |     |
|               | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W | R/W |     |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0   | 0   |     |
|               | 0    | 0    | 0    | 0    | 0    | 0    | 0   | 0   |     |

## Quản lý ngắt

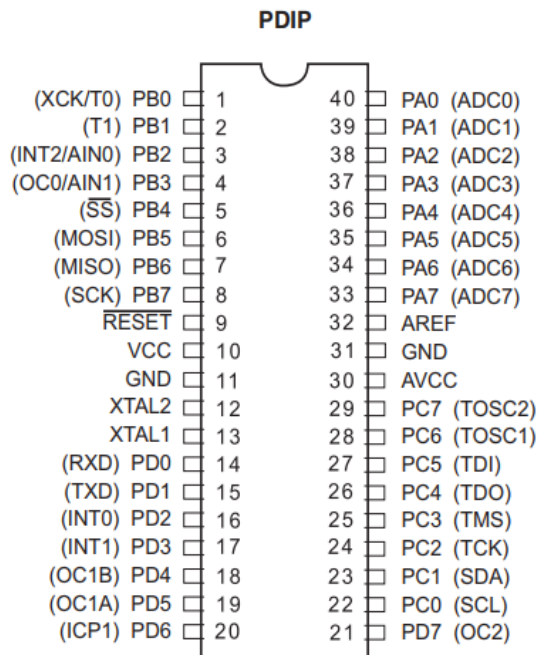
Khi có tín hiệu báo ngắt CPU sẽ tạm dừng công việc đang thực hiện lại và lưu vị trí đang thực hiện chương trình (con trỏ PC) vào ngăn xếp sau đó trỏ tới vector phục vụ ngắt và thực hiện chương trình phục vụ ngắt đó cho tới khi gặp lệnh RETI (Return from Interrupt) thì CPU lại lấy PC từ ngăn xếp ra và tiếp tục thực hiện chương trình mà trước khi có ngắt nó đang thực hiện.

Trong trường hợp mà có nhiều ngắt yêu cầu cùng một lúc thì CPU sẽ lưu các cờ báo ngắt đó lại, và thực hiện lần lượt các ngắt theo mức ưu tiên. Trong khi đang thực hiện ngắt mà xuất hiện ngắt mới thì sẽ xảy ra hai trường hợp. Trường hợp ngắt này có ưu tiên cao hơn thì nó sẽ được phục vụ, và ngược lại, nếu có mức ưu tiên thấp hơn thì nó sẽ bị bỏ qua.

Bộ nhớ ngăn xếp là vùng bất kì trong SRAM từ địa chỉ 0x60 trở lên. Để truy nhập vào SRAM thông thường thì ta dùng con trỏ X, Y, Z và truy cập vào SRAM theo kiểu ngăn xếp thì ta dùng con trỏ SP. Con trỏ này là một thanh ghi 16 bit và được truy cập như hai thanh ghi 8 bit chung có địa chỉ: SPL: 0x3D/0x5D (IO/SRAM) và SPH: 0x3E/0x5D.

Khi chương trình phục vụ ngắt hoặc chương trình con thì con trỏ PC được lưu vào ngăn xếp trong khi con trỏ ngăn xếp bị giảm hai vị trí. Và con trỏ ngăn xếp sẽ giảm 1 khi thực hiện lệnh PUSH. Ngược lại thực hiện lệnh POP thì con trỏ ngăn xếp sẽ tăng 1 và khi thực hiện lệnh RET hoặc RETI thì con trỏ ngăn xếp sẽ tăng 2. Như vậy, con trỏ ngăn xếp cần được chương trình đặt trước giá trị khởi tạo ngăn xếp trước khi một chương trình con được gọi hoặc các ngắt được cho phép phục vụ. Và giá trị ngăn xếp ít nhất cũng phải lớn hơn 60H (0x60) vì 5FH trở lại là vùng các thanh ghi

### 1.1.4. Các cổng I/O (xuất nhập)



Hình 1. 4 Cấu hình chân Atmega16

Các thanh ghi mô tả các cổng I/O:

#### Port A Data Register – PORTA

| Bit           | 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |       |
|---------------|--------|--------|--------|--------|--------|--------|--------|--------|-------|
|               | PORTA7 | PORTA6 | PORTA5 | PORTA4 | PORTA3 | PORTA2 | PORTA1 | PORTA0 | PORTA |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    |       |
| Initial Value | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |       |

#### Port A Data Direction Register – DDRA

| Bit           | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |      |
|---------------|------|------|------|------|------|------|------|------|------|
|               | DDA7 | DDA6 | DDA5 | DDA4 | DDA3 | DDA2 | DDA1 | DDA0 | DDRA |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  |      |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |      |

#### Port A Input Pins Address – PINA

| Bit           | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |      |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|------|
|               | PINA7 | PINA6 | PINA5 | PINA4 | PINA3 | PINA2 | PINA1 | PINA0 | PINA |
| Read/Write    | R     | R     | R     | R     | R     | R     | R     | R     |      |
| Initial Value | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   |      |

#### Port B Data Register – PORTB

| Bit           | 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |       |
|---------------|--------|--------|--------|--------|--------|--------|--------|--------|-------|
|               | PORTB7 | PORTB6 | PORTB5 | PORTB4 | PORTB3 | PORTB2 | PORTB1 | PORTB0 | PORTB |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    |       |
| Initial Value | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |       |

#### Port B Data Direction Register – DDRB

| Bit           | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |      |
|---------------|------|------|------|------|------|------|------|------|------|
|               | ddb7 | ddb6 | ddb5 | ddb4 | ddb3 | ddb2 | ddb1 | ddb0 | ddrb |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  |      |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |      |

#### Port B Input Pins Address – PINB

| Bit           | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |      |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|------|
|               | PINB7 | PINB6 | PINB5 | PINB4 | PINB3 | PINB2 | PINB1 | PINB0 | PINB |
| Read/Write    | R     | R     | R     | R     | R     | R     | R     | R     |      |
| Initial Value | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   |      |

**Port C Data Register – PORTC**

| Bit           | 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |       |
|---------------|--------|--------|--------|--------|--------|--------|--------|--------|-------|
|               | PORTC7 | PORTC6 | PORTC5 | PORTC4 | PORTC3 | PORTC2 | PORTC1 | PORTC0 | PORTC |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    |       |
| Initial Value | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |       |

**Port C Data Direction Register – DDRC**

| Bit           | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |      |
|---------------|------|------|------|------|------|------|------|------|------|
|               | DDC7 | DDC6 | DDC5 | DDC4 | DDC3 | DDC2 | DDC1 | DDC0 | DDRC |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  |      |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |      |

**Port C Input Pins Address – PINC**

| Bit           | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |      |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|------|
|               | PINC7 | PINC6 | PINC5 | PINC4 | PINC3 | PINC2 | PINC1 | PINC0 | PINC |
| Read/Write    | R     | R     | R     | R     | R     | R     | R     | R     |      |
| Initial Value | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   |      |

**Port D Data Register – PORTD**

| Bit           | 7      | 6      | 5      | 4      | 3      | 2      | 1      | 0      |       |
|---------------|--------|--------|--------|--------|--------|--------|--------|--------|-------|
|               | PORTD7 | PORTD6 | PORTD5 | PORTD4 | PORTD3 | PORTD2 | PORTD1 | PORTD0 | PORTD |
| Read/Write    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    | R/W    |       |
| Initial Value | 0      | 0      | 0      | 0      | 0      | 0      | 0      | 0      |       |

**Port D Data Direction Register – DDRD**

| Bit           | 7    | 6    | 5    | 4    | 3    | 2    | 1    | 0    |      |
|---------------|------|------|------|------|------|------|------|------|------|
|               | DDD7 | DDD6 | DDD5 | DDD4 | DDD3 | DDD2 | DDD1 | DDD0 | DDRD |
| Read/Write    | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  | R/W  |      |
| Initial Value | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 0    |      |

**Port D Input Pins Address – PIND**

| Bit           | 7     | 6     | 5     | 4     | 3     | 2     | 1     | 0     |      |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|------|
|               | PIND7 | PIND6 | PIND5 | PIND4 | PIND3 | PIND2 | PIND1 | PIND0 | PIND |
| Read/Write    | R     | R     | R     | R     | R     | R     | R     | R     |      |
| Initial Value | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   | N/A   |      |

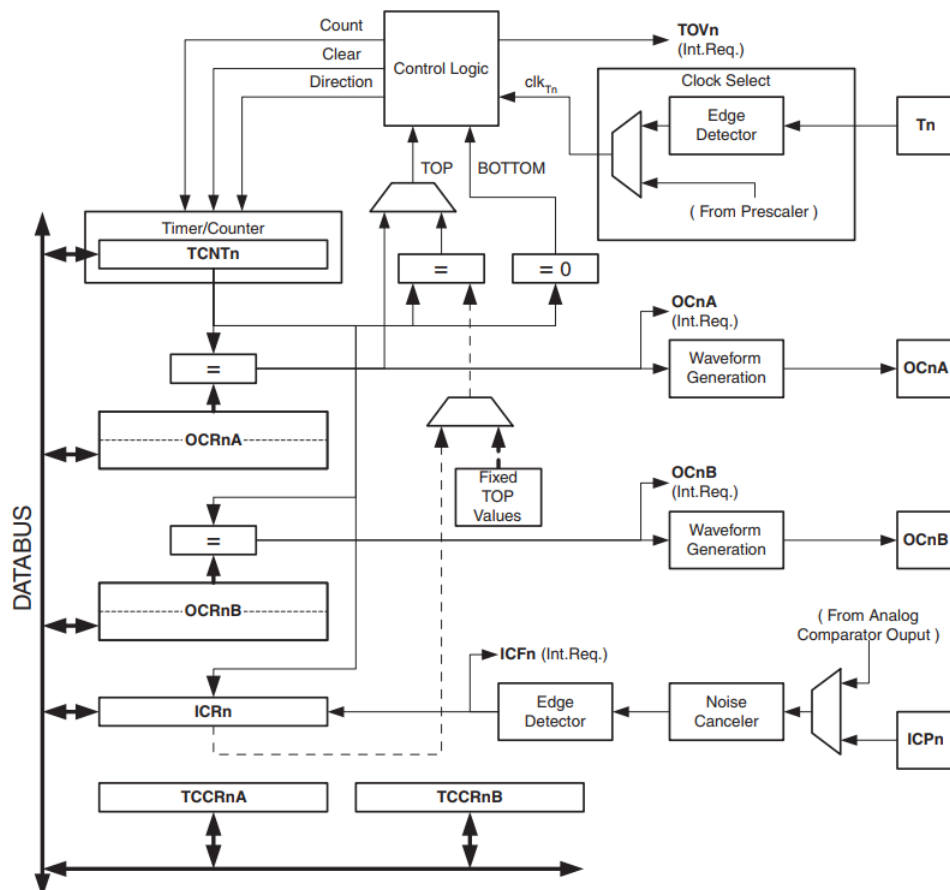
## Chức năng các chân:

- Chân VCC (chân số 10) là VCC cấp điện áp nguồn cho vi điều khiển. Nguồn điện cấp trong khoảng  $+5V \pm 0.5$ .
- Chân GND (chân số 11 và 31) nối đất (hay nối Mass). Khi thiết kế cần sử dụng một mạch ổn áp để bảo vệ cho vi điều khiển, các đơn giản nhất là dùng IC ổn áp 7805
- Port A (PA): Port A gồm 8 chân (từ chân 33 đến chân 40), có chức năng đầu vào cho chuyển đổi ADC.
- Port B (PB): Port B gồm 8 chân (từ chân 1 đến chân 8), ngoài có chức năng làm các đường xuất/nhập thì còn nhiều chức năng khác như giao tiếp SPI, so sánh điện áp tương tự đầu vào, tạo xung PWM timer 0, đếm xung ngoại
- Port C (PC): Port C gồm 8 chân (từ chân 22 đến chân 29) nếu giao tiếp JTAG được kích hoạt, điện trở trên các chân PC5 (TDI), PC3 (TMS), PC2 (TCK) sẽ được kích hoạt ngay cả khi khởi động lại (reset), ngoài ra còn có giao tiếp I2C
- Port D (PD): Port D gồm 8 chân (từ chân 14 đến 21): chức năng xuất nhập

- Chân RESET (RST): ngõ vào RST ở chân 9 là ngõ vào Reset dùng để thiết lập trạng thái ban đầu cho vi điều khiển. Hệ thống sẽ được thiết lập lại các giá trị ban đầu nếu ngõ này ở mức 1 tối thiểu 2 chu kì clock.
- Chân XTAL1 và XTAL2: Hai chân này có vị trí chân là 12 và 13 được sử dụng để nhận nguồn xung clock từ bên ngoài để hoạt động, thường được ghép nối với thạch anh và các tụ để tạo nguồn xung clock ổn định.
- Chân AVCC: Nguồn cấp cho cổng A và bộ chuyển đổi ADC, chân này được nối với nguồn VCC bên ngoài, ngay cả khi bộ chuyển đổi ADC không được sử dụng. Nếu bộ chuyển đổi ADC không được sử dụng, chân AVCC nên được nối với nguồn qua bộ lọc.
- Chân AREF: AREF là chân chuẩn analog cho bộ chuyển đổi ADC.

#### 1.1.5. Bộ định thời, bộ đếm Timer/Counter

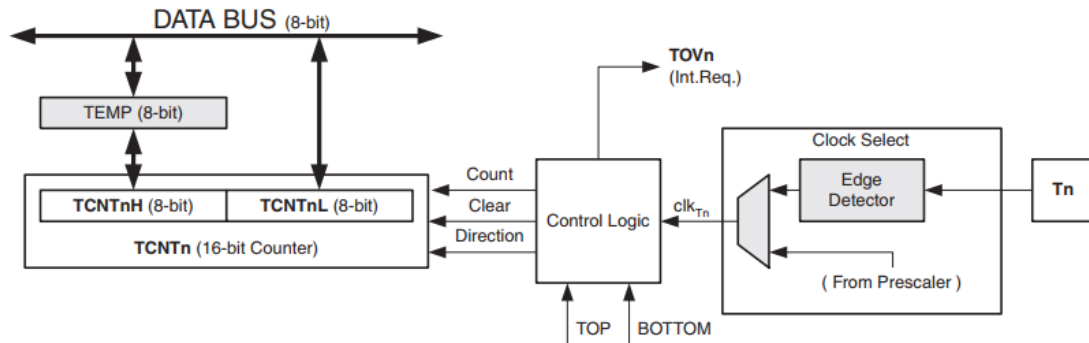
Atmega cung cấp cho lập trình viên 3 bộ timer/counter đó là: timer/counter0 8-bit, timer/counter1 16-bit, timer/counter 8-bit. Trong đề tài này chúng em sử dụng timer/counter1



Hình 1. 5 Cấu trúc khối timer/counter1

**Các thanh ghi chính:** Timer/Counter (TCNT1), thanh ghi so sánh :Output Compare Registers (OCR1A/B) và thanh ghi Input Capture Register (ICR1)

**Bộ đếm:** Phần chính của bộ định thời / bộ đếm 16 bit là bộ định hướng 16 bit có thể lập trình được bộ đếm



**Hình 1. 6 Khối bộ đếm**

**Count:** tăng hoặc giảm TCNT1 đi 1.

**Direction:** chọn tăng hay giảm.

**Clear:** xóa TCNT1 (đặt tất cả các bit về 0).

**clkT1:** Timer/Counter clock.

**TOP:** giá trị lớn nhất của bộ định thời/bộ đếm.

**BOTTOM:** giá trị nhỏ nhất của bộ định thời (0).

### Các chế độ hoạt động

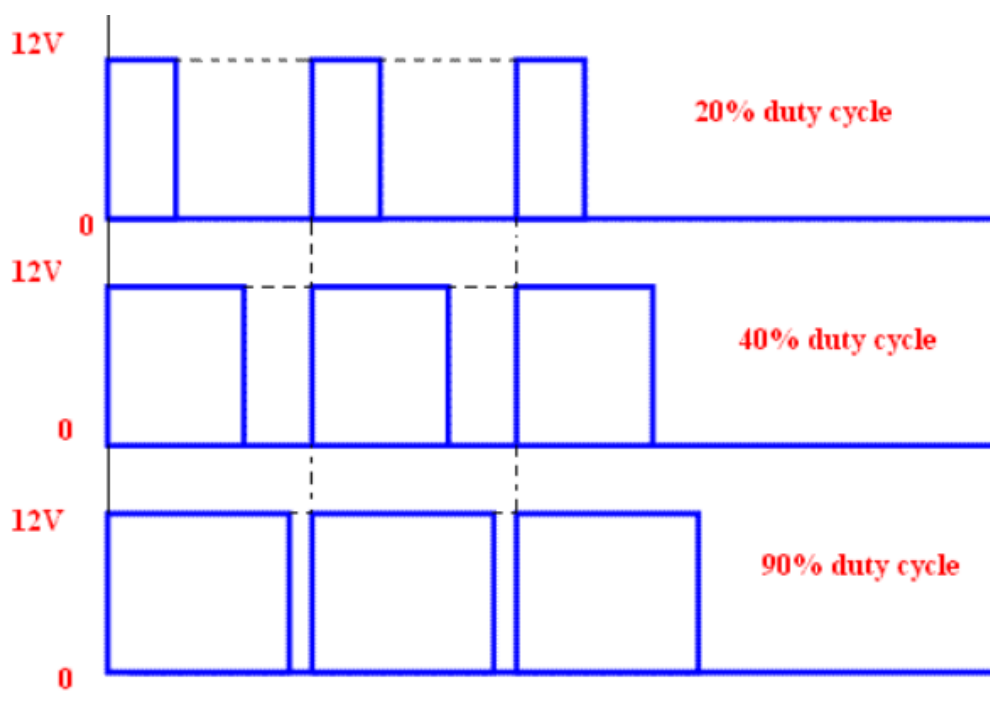
- Normal: đây là chế độ mặc định, bộ đếm sẽ đếm từ 0x0000 đến 0xFFFF
- CTC(compare match mode): xóa timer ở chế độ so sánh
- Fast PWM
- Phase Correct PWM Mode
- Phase and Frequency Correct PWM Mode

## 1.2. PWM(Pulse Width Modulation)- điều chế độ rộng xung

### 1.2.1. Tổng quan về PWM

Phương pháp điều xung PWM (Pulse Width Modulation) là phương pháp điều chỉnh điện áp ra tải, hay nói cách khác, là phương pháp điều chế dựa trên sự thay đổi độ rộng của chuỗi xung vuông, dẫn đến sự thay đổi điện áp ra.

Các PWM khi biến đổi thì có cùng 1 tần số và khác nhau về độ rộng của sườn âm và sườn dương



Hình 1. 7 Đồ thị dạng xung điều chế PWM

### Ứng dụng của PWM trong điều khiển

PWM được ứng dụng nhiều trong điều khiển. Điển hình nhất mà chúng ta thường hay gặp là điều khiển động cơ và các bộ xung áp, điều áp... Sử dụng PWM điều khiển độ nhanh chậm của động cơ hay cao hơn nữa, nó còn được dùng để điều khiển sự ổn định tốc độ động cơ.

Ngoài lĩnh vực điều khiển hay ổn định tải thì PWM còn tham gia và điều chế các mạch nguồn như : boot, buck, nghịch lưu 1 pha và 3 pha...

PWM còn gặp nhiều trong thực tế ở các mạch điện điều khiển. Điều đặc biệt là PWM chuyên dùng để điều khiển các phần tử điện tử công suất có đường đặc tính là tuyến tính khi có sẵn 1 nguồn 1 chiều cố định .Như vậy PWM được ứng dụng rất nhiều trong các thiết bị điện- điện tử. PWM cũng chính là nhân tố mà các đội Robocon sử dụng để điều khiển động cơ hay ổn định tốc độ động cơ.

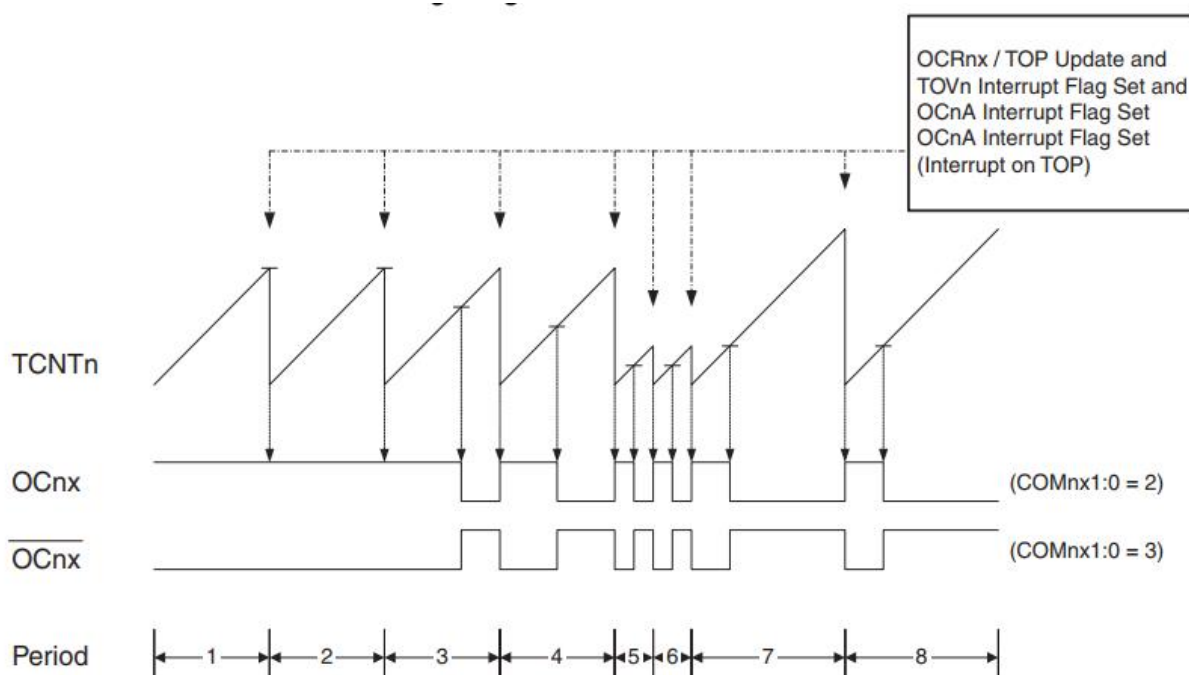
### 1.2.2. Điều chế xung PWM bằng chế độ Fast PWM của vi điều khiển Atmega16

Ở đề tài này chúng em dùng bộ timer/counter1 để điều chế xung PWM cụ thể là mode 14 đó là Fast PWM với giá trị TOP được ghi lên thanh ghi ICR1

| Mode | WGM13 | WGM12 (CTC1) | WGM11 (PWM11) | WGM10 (PWM10) | Timer/Counter Mode of Operation  | TOP    | Update of OCR1X | TOV1 Flag Set on |
|------|-------|--------------|---------------|---------------|----------------------------------|--------|-----------------|------------------|
| 0    | 0     | 0            | 0             | 0             | Normal                           | 0xFFFF | Immediate       | MAX              |
| 1    | 0     | 0            | 0             | 1             | PWM, Phase Correct, 8-bit        | 0x00FF | TOP             | BOTTOM           |
| 2    | 0     | 0            | 1             | 0             | PWM, Phase Correct, 9-bit        | 0x01FF | TOP             | BOTTOM           |
| 3    | 0     | 0            | 1             | 1             | PWM, Phase Correct, 10-bit       | 0x03FF | TOP             | BOTTOM           |
| 4    | 0     | 1            | 0             | 0             | CTC                              | OCR1A  | Immediate       | MAX              |
| 5    | 0     | 1            | 0             | 1             | Fast PWM, 8-bit                  | 0x00FF | BOTTOM          | TOP              |
| 6    | 0     | 1            | 1             | 0             | Fast PWM, 9-bit                  | 0x01FF | BOTTOM          | TOP              |
| 7    | 0     | 1            | 1             | 1             | Fast PWM, 10-bit                 | 0x03FF | BOTTOM          | TOP              |
| 8    | 1     | 0            | 0             | 0             | PWM, Phase and Frequency Correct | ICR1   | BOTTOM          | BOTTOM           |
| 9    | 1     | 0            | 0             | 1             | PWM, Phase and Frequency Correct | OCR1A  | BOTTOM          | BOTTOM           |
| 10   | 1     | 0            | 1             | 0             | PWM, Phase Correct               | ICR1   | TOP             | BOTTOM           |
| 11   | 1     | 0            | 1             | 1             | PWM, Phase Correct               | OCR1A  | TOP             | BOTTOM           |
| 12   | 1     | 1            | 0             | 0             | CTC                              | ICR1   | Immediate       | MAX              |
| 13   | 1     | 1            | 0             | 1             | Reserved                         | –      | –               | –                |
| 14   | 1     | 1            | 1             | 0             | Fast PWM                         | ICR1   | BOTTOM          | TOP              |
| 15   | 1     | 1            | 1             | 1             | Fast PWM                         | OCR1A  | BOTTOM          | TOP              |

**Hình 1. 8 Các chế độ hoạt động của timer/counter1**

**Fast PWM:**



**Hình 1. 9 Mô tả hoạt động Fast PWM**

Fast PWM khác với PWM ở chỗ Fast PWM là kiểu 1 sườn dốc(single-slope). Thanh ghi TCNTn có thể đếm từ BOTTOM(0x0000) đến TOP(0xFFFF) sau đó cờ tràn được bật TCNTn được đặt về BOTTOM hoặc giá trị TOP có thể được đặt bởi ICR1. Mỗi khi đến giá trị so sánh giá trị thanh ghi OCnx sẽ đảo trạng thái.

Bộ Timer1 của Atmega16 cung cấp cho chúng ta 2 kênh tạo xung điều rộng PWM:

- Kênh A: tín hiệu ra ở OC1A (PORTB.1)
- Kênh B: tín hiệu ra ở OC1B (PORTB.2)

Trong đồ án này, sử dụng chế độ Fast PWM, mode 14 (Kênh A).

- Thiết lập dạng tín hiệu PWM ra trên thanh ghi TCCR1A .
- Khi hoạt động, ban đầu chân OC1A ở mức cao (tùy vào dạng tín hiệu PWM ra chọn ở trên), TCNT1 tăng giá trị từ 0 cho đến khi bằng giá trị trên thanh ghi OCR1A26thì chân OC1A được xóa về 0. TCNT1 vẫn tiếp tục tăng đến khi bằng giá trị trong thanh ghi ICR1 thì TCNT1 reset về 0 và chân OC1A được kéo lên mức cao.

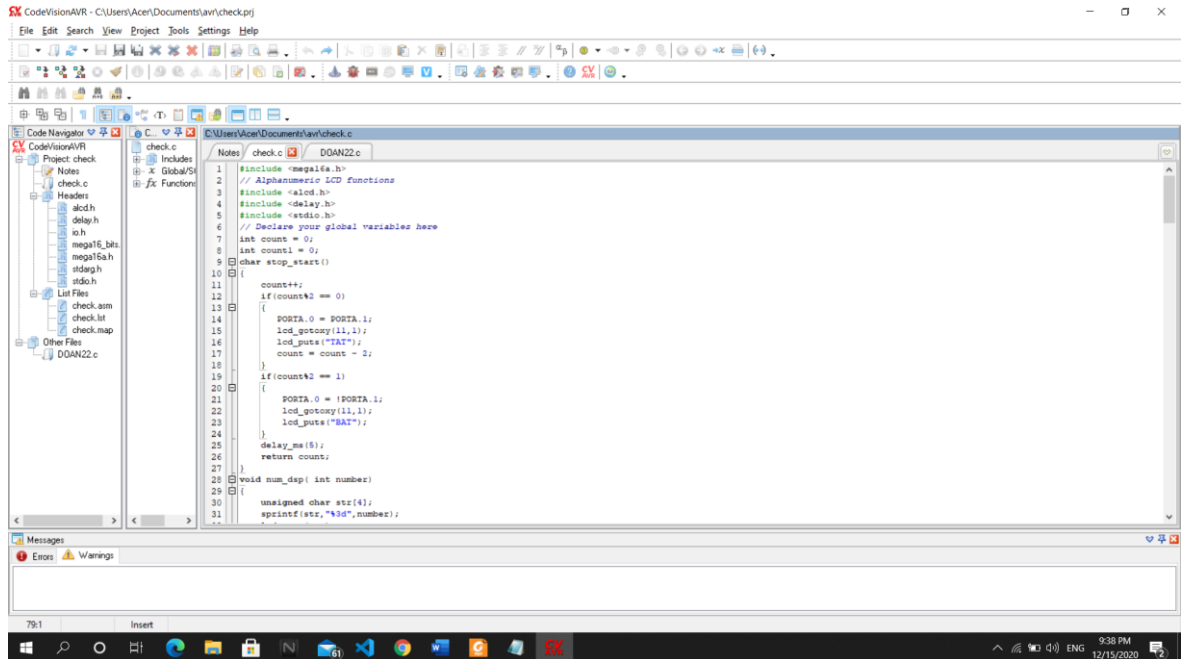
Thiết lập:

- Giá trị của ICR1: là chu kỳ xung.
- Giá trị của OCR1A: là thời gian xung ở mức cao.

## 1.3. Các phần mềm sử dụng

### 1.3.1. Trình biên dịch CodevisionAVR

CodeVisionAVR là một trình biên dịch chéo C, môi trường phát triển tích hợp và tạo chương trình tự động được thiết kế cho họ AVR của Atmel

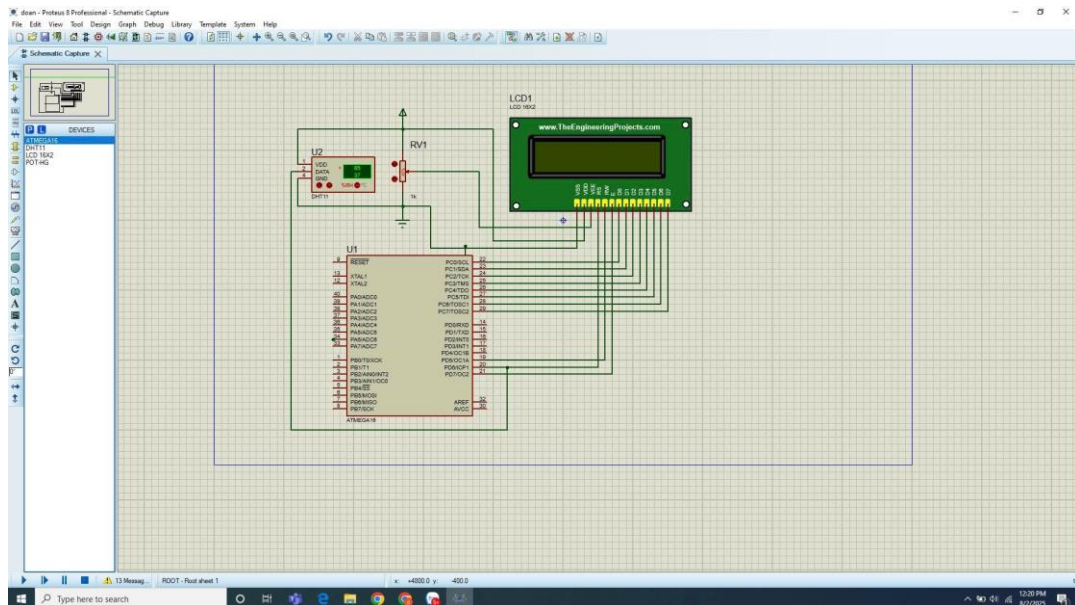


Hình 1. 10 Giao diện của CodevisionAVR

- Ưu điểm: CodevisionAVR cung cấp nhiều thư viện khởi tạo cho người lập trình, giúp lập trình viên dễ dàng trong việc chạy code và debug
- Nhược điểm: sau khi khởi tạo thì khá nhiều phần rườm rà, việc sử dụng thư viện dẫn đến người lập trình không hiểu hết chức năng của vi điều khiển gây khó khăn trong debug mạch thực tế

### 1.3.2. Phần mềm mô phỏng Proteus

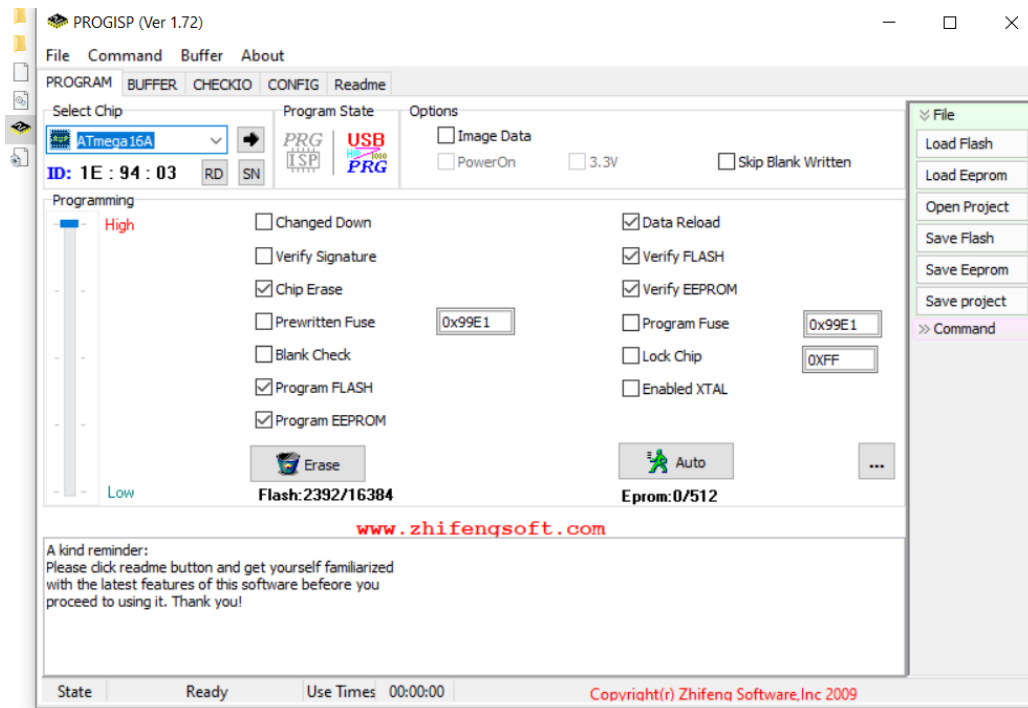
Proteus giúp người dùng mô phỏng mạch, kiểm tra code để sau đó dựa vào kết quả mô phỏng mà ta đi thiết kế mạch thực tế



Hình 1. 11 Giao diện Proteus

### 1.3.3. Phần mềm nạp code ProgISP

Phần mềm giúp người lập trình nạp code viết từ trình biên dịch CodevisionAVR đã chạy mô phỏng ở proteus xuống kit nạp Atmega16. Lưu ý cần cấu hình Fulse Bit lại để PORTC được sử dụng đúng mục đích do chức năng JTAG của vi điều khiển đã cấu hình PORTC

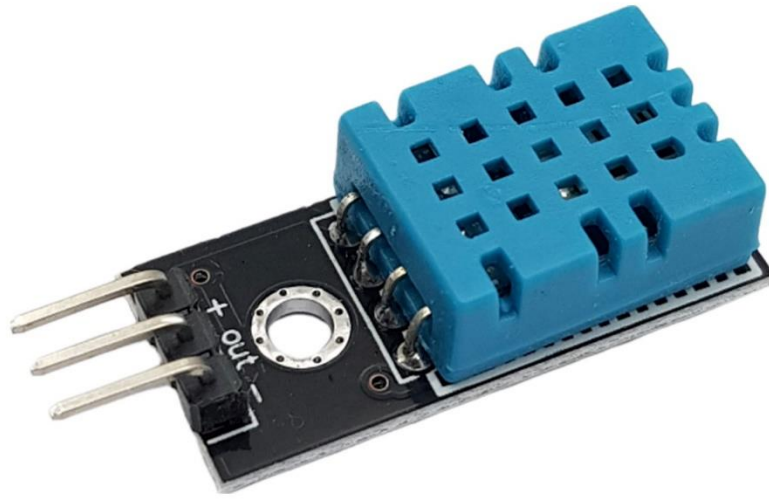


**Hình 1. 12 Giao diện ProgISP 1.72**

## **1.4. Các linh kiện được sử dụng**

### **1.4.1. Cảm biến DHT11**

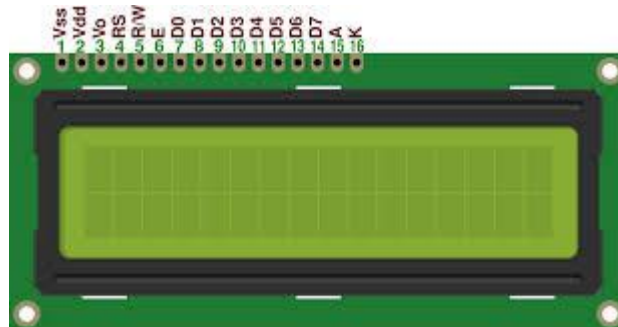
- Điện áp hoạt động: 3V - 5.5V DC.
- Dòng điện tiêu thụ: 0.3mA (đo), 60uA (chế độ chờ).
- Dải đo độ ẩm: 5% - 95% RH.
- Dải đo nhiệt độ: -20°C - 60°C.
- Cảm biến DHT11 có 3 chân, mỗi chân có chức năng riêng biệt. Các chân bao gồm: VCC (cấp nguồn), GND (nối đất), S (tín hiệu ngõ ra).



**Hình 1. 13 Cảm biến DHT11**

#### **1.4.2. Màn hình hiển thị LCD**

Đây là loại màn hình phổ biến dùng chip HD7470, hiển thị 16 kí tự 2 hàng

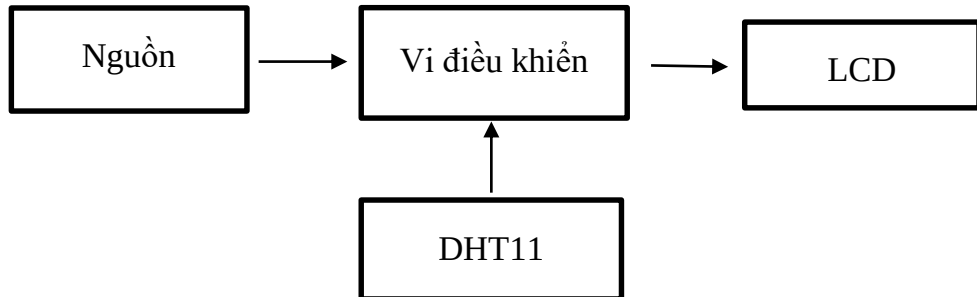


**Hình 1. 14 LCD 16x2**

- Vss: cấp GND
- Vdd: cấp +5V
- Vo: chỉnh độ đậm nhạt
- RS, R/W, E: các chân điều khiển
- D0-D7: chân dữ liệu
- A,K(đèn nền): cấp +5V và GND

## CHƯƠNG 2: THIẾT KẾ MẠCH

### 2.1. Sơ đồ khối

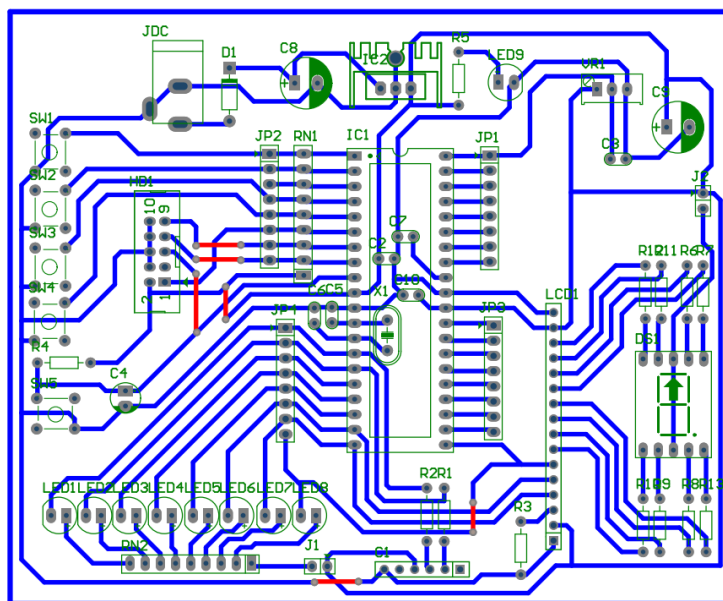


Hình 2. 1 Sơ đồ khối

Sơ đồ khối gồm 4 khối chính:

- khối nguồn: nguồn vào cung cấp cho cả vi điều khiển
- khối input: cảm biến DHT11 gửi tín hiệu đầu vào
- khối xử lí: vi điều khiển atmega xử lí đầu vào, tính toán các thông số sau đó gửi ra output
- khối output: LCD hiển thị các thông số nhiệt độ và độ ẩm của môi trường

### 2.2. Mạch in



Hình 2. 2 Mạch in PCB

## CHƯƠNG 3: YÊU CẦU MẠCH

### 3.1. Yêu cầu chức năng

Mạch thiết kế có chức năng chính là **đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến DHT11**, sau đó **hiển thị kết quả lên màn hình LCD 16x2**. Vi điều khiển trung tâm sử dụng trong mạch là **ATmega16**, hoạt động ở tần số xung nhịp 16 MHz.

Các chức năng cụ thể của mạch bao gồm:

**Giao tiếp với cảm biến DHT11** thông qua một chân dữ liệu duy nhất (1-wire), trong đó vi điều khiển chủ động gửi tín hiệu khởi tạo và nhận chuỗi dữ liệu 40 bit từ cảm biến, bao gồm thông tin về độ ẩm và nhiệt độ.

**Xử lý chuỗi dữ liệu** nhận được từ DHT11 để tách riêng giá trị độ ẩm và nhiệt độ, sau đó chuyển đổi thành định dạng số nguyên hoặc số thực có thể hiển thị.

### 3.2. Yêu cầu phi chức năng

Mạch chạy cần ổn định, đáp ứng nhanh các thao tác của người sử dụng, ngoài ra mạch cần phải gọn gàng ít kết nối dây nhiều đảm bảo tính thẩm mỹ cho mạch

### 3.3. Ứng dụng

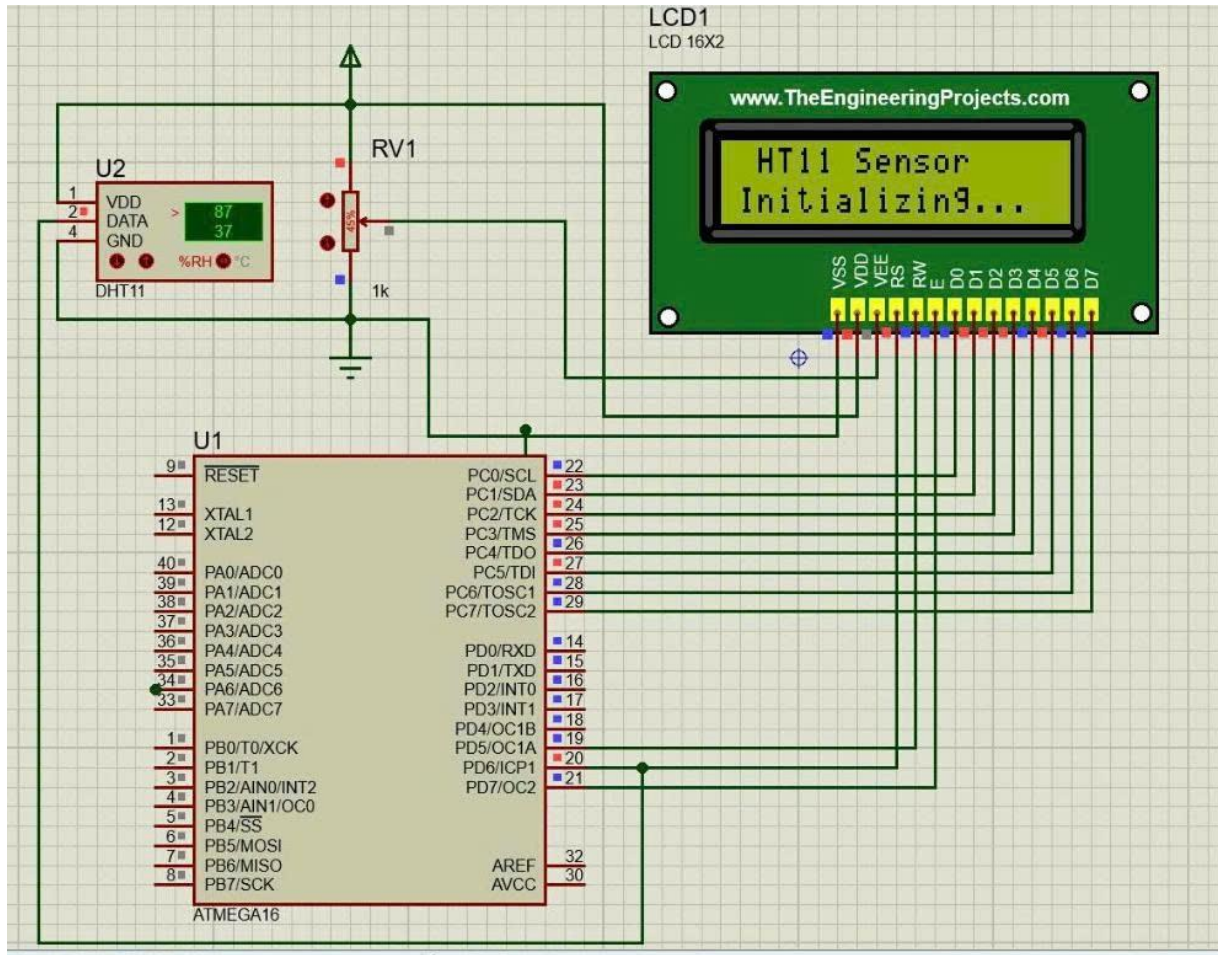
Mạch đo nhiệt độ và độ ẩm sử dụng vi điều khiển ATmega16 kết hợp với cảm biến DHT11 có nhiều ứng dụng thực tiễn trong đời sống và công nghiệp. Với khả năng thu thập và hiển thị thông tin môi trường theo thời gian thực, mạch có thể được sử dụng trong các hệ thống như:

- **Hệ thống giám sát môi trường** trong nhà ở, văn phòng, nhà kho, hoặc phòng thí nghiệm để đảm bảo điều kiện nhiệt ẩm luôn trong mức cho phép.
- **Hệ thống nhà thông minh (Smart Home)**: mạch có thể tích hợp vào hệ thống điều khiển tự động điều hòa, quạt, hoặc máy hút ẩm nhằm tiết kiệm năng lượng và nâng cao sự thoải mái cho người dùng.
- **Nông nghiệp công nghệ cao**: theo dõi nhiệt độ và độ ẩm trong nhà kính, chuồng trại, giúp người vận hành điều chỉnh các điều kiện lý tưởng cho cây trồng và vật nuôi.
- **Ứng dụng giáo dục**: mạch đơn giản, dễ chế tạo, rất phù hợp cho sinh viên, học sinh trong các môn học vi điều khiển, hệ thống nhúng, hoặc các khóa học về điện tử ứng dụng.
- **Thiết bị đo đạc cầm tay**: nhờ kích thước nhỏ gọn, mạch có thể được dùng làm thiết bị đo nhanh tại hiện trường với hiển thị trực tiếp qua màn hình LCD.

## CHƯƠNG 4: KẾT QUẢ THỰC HIỆN

### 4.1. Kết quả mô phỏng

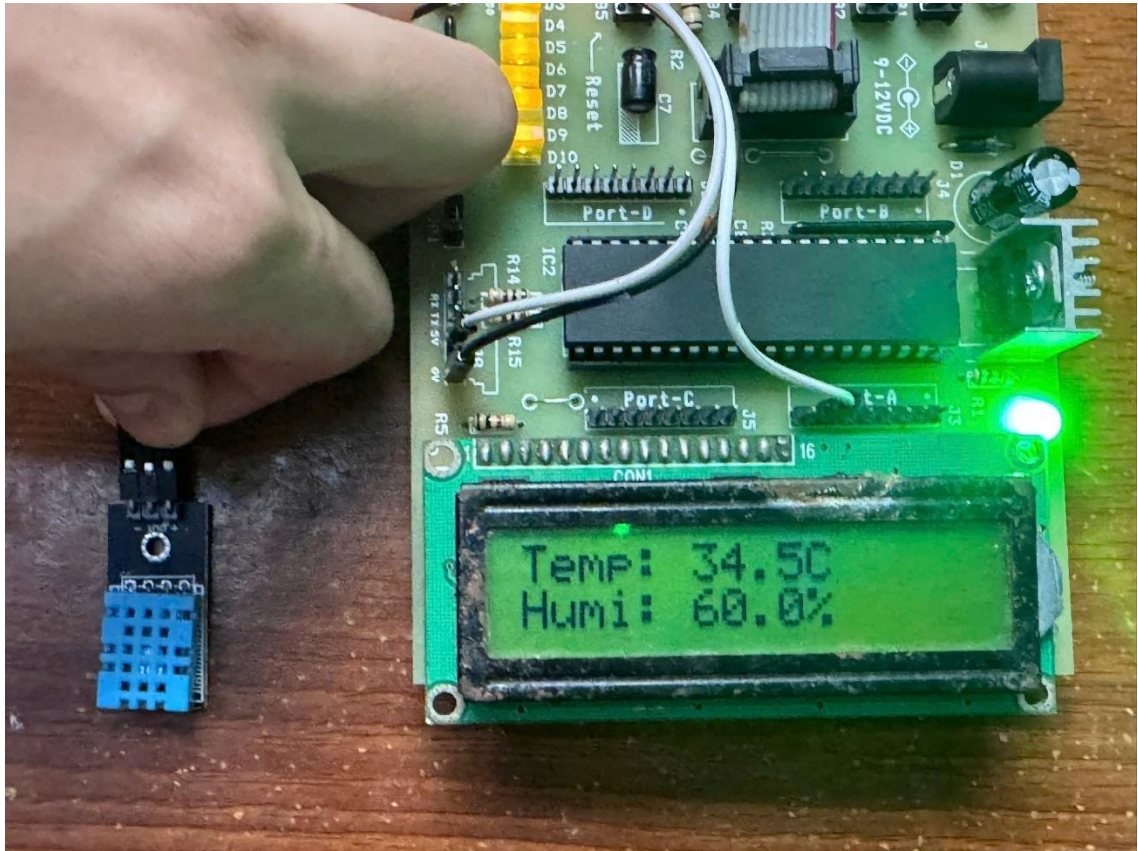
Kết quả mô phỏng chạy đúng một phần với yêu cầu đề ra



Hình 4. 1 Kết quả mô phỏng bằng phần mềm Proteus

### 4.2. Kết quả thực tế trên mạch

Tuy không thể đạt thông số như yêu cầu và mạch mô phỏng nhưng mạch thực tế chạy khá tốt và đạt yêu cầu đề ra



Hình 4. 2 Kết quả thực tế

## KẾT LUẬN

Sau một kỳ tìm hiểu em đã cơ bản hoàn thành đề tài “**Thiết kế hệ thống đo nhiệt độ độ ẩm**”. Trong quá trình xây dựng và hoàn thành sản phẩm, em đã tích lũy được kinh nghiệm thiết kế, làm và hàn mạch. Trong quá trình thực hiện đề tài, do còn nhiều hạn chế về mặt thời gian, kiến thức cũng như công cụ và kinh nghiệm nên đề tài vẫn còn những thiếu sót nhất định, mong được thầy và các bạn đánh giá, góp ý một cách khách quan để em có thể rút ra kinh nghiệm và hoàn thiện đề tài hơn nữa.

Để hoàn thành bài báo cáo này, em xin chân thành gửi lời cảm ơn tới sự quan tâm và hướng dẫn tận tình của thầy!

# PHỤ LỤC

## PHỤ LỤC 1: SOURCE CODE

### ***File <main.c>***

```
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <string.h>
#include "LCD.h"
#include "DHT11.h"

int main(void) {
    float humidity, temperature;
    char buffer[16];

    // Initialize LCD
    LCD_Init();

    // Display welcome message
    LCD_String("DHT11 Sensor");
    LCD_Gotoxy(0, 1);
    LCD_String("Initializing...");
    _delay_ms(2000);
    LCD_Clear();

    while (1) {
        // Read data from DHT11
        DHT11_Read_Data(&humidity, &temperature);

        // Clear display
        LCD_Clear();

        // Check if reading was successful
        if (humidity != -1 && temperature != -1) {
            // Display temperature on first line
            LCD_Gotoxy(0, 0);
            LCD_String("Temp: ");
            dtostrf(temperature, 4, 1, buffer); // Convert float to
string (4 digits, 1 decimal)
            LCD_String(buffer);
            LCD_String("C");

            // Display humidity on second line
            LCD_Gotoxy(0, 1);
            LCD_String("Humi: ");
            dtostrf(humidity, 4, 1, buffer); // Convert float to
string (4 digits, 1 decimal)
            LCD_String(buffer);
            LCD_String("%");
        } else {
            // Display error message
            LCD_Gotoxy(0, 0);
            LCD_String("Sensor Error!");
            LCD_Gotoxy(0, 1);
            LCD_String("Check Connection");
        }
    }
}
```

```

    }

    // Wait 3 seconds before next reading
    _delay_ms(3000);
}

return 0;
}

```

### ***File <LCD.h>***

```

#ifndef LCD_H_
#define LCD_H_

#include <avr/io.h>
#include <util/delay.h>

// Define LCD control pins
#define LCD_RS PD6
#define LCD_RW PD5
#define LCD_EN PD7

// Define LCD data port
#define LCD_DATA_PORT PORTC
#define LCD_DATA_DDR DDRC
#define LCD_CTRL_PORT PORTD
#define LCD_CTRL_DDR DDRD

// Function declarations only
void LCD_Command(unsigned char cmd);
void LCD_Char(unsigned char data);
void LCD_Init(void);
void LCD_String(const char *str);
void LCD_Clear(void);
void LCD_Gotoxy(const unsigned char x, const unsigned char y);
void LCD_Puts(const char *str);
void LCD_PutChar(const char data);

#endif /* LCD_H_ */

```

## ***File <DHT11.c>***

```
#include "DHT11.h"

void DHT11_Request(void) {
    DDRA |= (1 << DHT11_PIN); // Set DHT11 pin as output
    PORTA &= ~(1 << DHT11_PIN); // Pull the pin low
    _delay_ms(18); // Wait for 18ms
    PORTA |= (1 << DHT11_PIN); // Pull the pin high
    _delay_us(20); // Wait for 20us
    DDRA &= ~(1 << DHT11_PIN); // Set DHT11 pin as input
}

void DHT11_Response(void) {
    while(PINA & (1 << DHT11_PIN)); // Wait for the pin to go low
    while(!(PINA & (1 << DHT11_PIN))); // Wait for the pin to go high
    while(PINA & (1 << DHT11_PIN)); // Wait for the pin to go low
}

uint8_t DHT11_ReceiveData(void) {
    uint8_t data = 0;
    for (int i = 0; i < 8; i++) {
        while(!(PINA & (1 << DHT11_PIN))); // Wait for the pin to go high
        _delay_us(30); // Wait for 30us
        if (PINA & (1 << DHT11_PIN)) {
            data = (data << 1) | 0x01; // Read 1
        } else {
            data = (data << 1); // Read 0
        }
        while(PINA & (1 << DHT11_PIN)); // Wait for the pin to go low
    }
    return data;
}

void DHT11_Read_Data(float* humidity, float* temperature) {
    uint8_t Rh_byte1, Rh_byte2, Temp_byte1, Temp_byte2, sum;
    DHT11_Request();
    DHT11_Response();
    Rh_byte1 = DHT11_ReceiveData();
    Rh_byte2 = DHT11_ReceiveData();
    Temp_byte1 = DHT11_ReceiveData();
    Temp_byte2 = DHT11_ReceiveData();
    sum = DHT11_ReceiveData();

    if (sum == (Rh_byte1 + Rh_byte2 + Temp_byte1 + Temp_byte2)) {
        *humidity = (float)Rh_byte1 + (float)Rh_byte2 / 10.0;
        *temperature = (float)Temp_byte1 + (float)Temp_byte2 / 10.0;
    } else {
        // Handle checksum error
        *humidity = -1;
        *temperature = -1;
    }
}
```

### **File <LCD.c>**

```
#include "LCD.h"

// Send a command to the LCD
void LCD_Command(unsigned char cmd) {
    LCD_DATA_PORT = cmd; // Put command on data port
    LCD_CTRL_PORT &= ~(1 << LCD_RS); // RS=0 for command register
    LCD_CTRL_PORT |= (1 << LCD_EN); // Enable pulse
    _delay_us(1);
    LCD_CTRL_PORT &= ~(1 << LCD_EN); // Disable pulse
    _delay_ms(2); // Wait for command to execute
}

// Send data to the LCD
void LCD_Char(unsigned char data) {
    LCD_DATA_PORT = data; // Put data on data port
    LCD_CTRL_PORT |= (1 << LCD_RS); // RS=1 for data register
    LCD_CTRL_PORT |= (1 << LCD_EN); // Enable pulse
    _delay_us(1);
    LCD_CTRL_PORT &= ~(1 << LCD_EN); // Disable pulse
    _delay_ms(2); // Wait for data to write
}

// Initialize the LCD
void LCD_Init(void) {
    LCD_CTRL_DDR = 0xFF; // Set control port as output
    LCD_DATA_DDR = 0xFF; // Set data port as output
    _delay_ms(20); // Wait for LCD to power up
    LCD_Command(0x38); // Function set: 8-bit mode, 2 line, 5x7 dots
    LCD_Command(0x0C); // Display on, cursor off
    LCD_Command(0x06); // Entry mode set: increment cursor
    LCD_Command(0x01); // Clear display screen
    _delay_ms(2); // Wait for command to execute
}

// Display a string on the LCD
void LCD_String(const char *str) {
    while (*str) {
        LCD_Char(*str++);
    }
}

// Clear the LCD display
void LCD_Clear(void) {
    LCD_Command(0x01); // Clear display
    _delay_ms(2); // Wait for command to execute
    LCD_Command(0x80); // Move cursor to home position
}

// Set cursor position
void LCD_Gotoxy(const unsigned char x, const unsigned char y) {
    const unsigned char address[] = {0x80, 0xC0, 0x94, 0xD4}; // Base
    addresses for each line
    LCD_Command(address[y] + x); // Set DDRAM address
}

// Display a string on the LCD (alternative name)
void LCD_Puts(const char *str) {
    while (*str != '\0') {

```

```

        LCD_Char(*str++);
    }
}

// Display a single character on the LCD (alternative name)
void LCD_PutChar(const char data) {
    LCD_Char(data);
}

```

### ***File <DHT11.h>***

```

#ifndef DHT11_H_
#define DHT11_H_

#include <avr/io.h>
#include <util/delay.h>

// Define the data pin connected to DHT11
#define DHT11_PIN PA6 // Changed from PA1 to PA6

// Function declarations only
void DHT11_Request(void);
void DHT11_Response(void);
uint8_t DHT11_ReceiveData(void);
void DHT11_Read_Data(float* humidity, float* temperature);

#endif /* DHT11_H_ */

```